

Slovak Technical University in Bratislava
Faculty of Informatics and Information Technologies

FIIT-127326

Adam Svitek

**Unified Interoperability: Bridging
Homogeneous and Heterogeneous
Blockchains**

Bachelor Thesis

Supervisor: Ing. Dušan Morháč

Máj 2025

Slovak Technical University in Bratislava
Faculty of Informatics and Information Technologies

FIIT-127326

Adam Svitek

**Unified Interoperability: Bridging
Homogeneous and Heterogeneous
Blockchains**

Bachelor Thesis

Study Program: Software Engineering

Degree Course: 9.2.5 Software Engineering

Place of Work: Institute of Informatics and Software Engineering, FIIT STU,
Bratislava

Supervisor: Ing. Dušan Morháč

Máj 2025

Annotation

Slovak University of Technology Bratislava

Faculty of Informatics and Information Technologies

Degree Course: Software Engineering

Author: Adam Svitek

Bachelor Thesis: Unified Interoperability: Bridging Homogeneous and Heterogeneous Blockchains

Supervisor: Ing. Dušan Morháč

Máj 2025

Acknowledgements

The acknowledgements and the people to thank go here, don't forget to include your project advisor(Ing. Dušan Morháč)...

Contents

1	Technical abstract	1
2	Lay summary	3
3	Introduction	5
4	Problem statement and solution	7
4.1	Problem statement	7
4.2	Technical literature review	8
4.2.1	Approaches to blockchain interoperability	8
4.2.2	Interoperability inside the Ethereum ecosystem	9
4.2.3	Interoperability between Ethereum and Polkadot	9
4.2.4	Related interoperability solutions	10
4.2.5	Summary of limitations	11
4.3	Solution overview (high level)	11
4.3.1	Main idea of the solution	12
4.3.2	Design logic	13
4.3.3	Supported transfer scenarios	13
4.3.4	Workflow and architecture overview	14
4.3.5	Limitations of the proposed solution	14

4.4	Risk assessment	15
4.4.1	Risk of wrong transfer plan	15
4.4.2	Risk of wrong address format	16
4.4.3	Risk of external dependencies	16
4.4.4	Risk of multi-step transfers	16
4.4.5	Risk of user error	17
4.5	Experimental reproducibility and integration	17
4.5.1	Implementation environment	17
4.5.2	Configuration and reproducibility	19
4.5.3	Protocol integration	20
4.5.4	Wallet and network integration	21
4.5.5	Reproducibility of test scenarios	22
4.5.6	Limitations of reproducibility	23
4.6	Sustainability and environmental impact	24
4.6.1	Technical sustainability	24
4.6.2	Environmental impact	25
4.7	Employability	26
4.8	Teamwork, diversity and inclusion	27
5	Conclusion	29
6	References	33
7	List of figures	35
A	Work plan	39
A.1	Winter semester	39
A.2	Summer semester	40
A.3	Evaluation of work plan	41

B	Installation and User Guide	43
B.1	Installation overview	43
B.2	Required software	43
B.3	Downloading the project	44
B.3.1	From GitHub repository	44
B.3.2	From ZIP archive	44
B.4	Installing dependencies	45
B.5	RPC endpoint configuration	45
B.6	Running the application	46
B.7	Wallet setup	47
B.8	Testnet tokens	47
B.9	User guide and testing workflow	48
B.9.1	Transfer page overview	48
B.9.2	EVM to EVM transfer	49
B.9.3	Ethereum Layer 1 to Asset Hub transfer	51
B.9.4	Ethereum Layer 2 to Asset Hub transfer	52
B.9.5	Westend Asset Hub to Ethereum Layer 1 transfer	54
B.9.6	Westend Asset Hub to Ethereum Layer 2 transfer	56
C	Digital media contents	59
D	Contents of Included CD-ROM	63

Chapter 1

Technical abstract

Modern blockchain ecosystems are divided across multiple networks, execution layers, and interoperability mechanisms. This helps with scalability, but it also makes asset transfers more difficult for users. A transfer that looks simple from the user's point of view may require several technical steps, especially when funds need to move between Ethereum Layer 1, Ethereum Layer 2 networks, and external ecosystems such as Polkadot.

Existing interoperability protocols already solve important parts of this problem. Some protocols focus on transfers within the Ethereum ecosystem, while others enable communication between Ethereum and other blockchain networks. However, these solutions are usually provided as separate tools. In more complex cases, the user still has to choose the correct protocol, switch between different applications, and execute the required steps in the correct order. This increases the complexity of the process and creates space for mistakes.

The goal of this thesis was to design and implement a web-based application that reduces this fragmentation at the application layer. The proposed solution does

not create a new bridge or interoperability protocol. Instead, it combines existing protocols into one guided transfer workflow. One protocol is used for transfers within the Ethereum ecosystem, while other is used for transfers between Ethereum and Polkadot.

The application was implemented as a client-side solution. It communicates with protocol APIs, blockchain RPC endpoints, and the user's wallet. Based on the selected source network, destination network, asset, amount, and recipient address, the application prepares a transfer plan and executes the required steps in sequence.

The implemented solution was evaluated in testnet environments using several transfer scenarios. The results show that the application mainly improves workflow coordination rather than protocol-level transfer speed. Its main contribution is the simplification of multi-step blockchain asset transfers through a single user-facing interface.

Chapter 2

Lay summary

Moving digital assets between different blockchain networks can be difficult for ordinary users. When a user wants to send tokens or cryptocurrency from one network to another, the process is often not completed in a single place. The user may need to open several tools, choose the correct transfer path, understand which interoperability tool should be used, and approve more than one transaction in the correct order. This can be confusing, especially when the transfer goes across different ecosystems such as Ethereum and Polkadot.

This thesis focuses on simplifying this process. The result of the work is a web application that guides the user through the transfer step by step. Instead of requiring the user to manually search for the right tools and decide how the transfer should be executed, the application collects only the necessary information: the source network, the destination network, the asset, the amount, and the recipient address. Based on this input, the application prepares the required transfer steps and presents the process through one unified interface. The final transactions are still approved by the user in their own wallet, so the application does not take control of the user's funds.

The proposed application does not make the blockchain transfer itself faster, because it still depends on the existing networks and protocols. However, it can make the whole process faster for the user. The user does not have to search for the right tools, decide which bridge to use, or manually plan the next step. By combining the process into one guided workflow, the application can reduce the time and effort and lower the chance of mistakes.

Chapter 3

Introduction

Blockchains have developed over time from isolated decentralized networks to larger ecosystems composed of multiple layers, protocols and environments. One example of this development is Ethereum, where users can interact not only with the main Layer 1 network, but also with several Layer 2 networks created to improve scalability and reduce higher transaction costs. At the same time, other ecosystems such as Polkadot use different architectures and provide their own ways of organizing assets, applications, and communication between chains.

This ongoing development also creates a practical problem for users who need to move digital assets between these environments. Although many interoperability protocols and bridges already exist, they often focus only on specific part of the transfer path. A user may be able to move assets between Ethereum Layer 2 networks and Ethereum Layer 1, but transferring the same assets further to another ecosystem may require a different tool and another transaction process. As a result, more complex transfers are still fragmented across many different interfaces and require manual decisions from the user, which can be a setback for some users.

The problem is therefore not only technical interoperability itself, but also the way interoperability is presented and coordinated for the user. To make such transactions, users must understand which protocol should be used, which network should be selected and in what order the transactions should be executed. This increases the difficulty of the whole process and creates a higher risk of mistakes.

This thesis focuses on reducing this fragmentation at the application layer. Instead of designing a new bridge or protocol, the work proposes a web application that combines selected existing interoperability mechanisms into one guided transfer workflow. The main goal is to make existing transfer routes easier to access, simplify route selection, support transfer planning, and provide step-by-step execution for selected transfer scenarios between Ethereum-based networks and the Polkadot ecosystem.

Chapter 4

Problem statement and solution

4.1 Problem statement

The problem addressed in this thesis is the fragmentation of asset transfers across different blockchain networks, layers, and ecosystems. Some blockchain networks may share similar technical properties, such as wallet interaction, address format, smart contract execution, or token standards, while others are based on different architectures. This thesis focuses mainly on Ethereum-based networks and the Polkadot ecosystem. Ethereum consists of the main Layer 1 network and several Layer 2 networks, which are closely related to Ethereum and often share similar user interaction patterns. Polkadot, on the other hand, is a different ecosystem with a different architecture, addressing model, and communication mechanisms between chains.

Users may need to move assets across these environments, for example, from an Ethereum Layer 2 network to Polkadot or in the opposite direction. Many interoperability solutions nowadays support many individual transfer paths, they often do not provide one workflow for more difficult routes. Some protocols are better

for transfers inside the Ethereum ecosystem, while another protocols are better for transfers between Ethereum and Polkadot. As a result, more difficult transfers may require more dependent steps, correct execution order, and the use of more than one web application.

This create some difficulties for the user. The user must firstly identify the correct route to be used, choose the suitable bridge or protocol, use different interfaces, enter the correct address format where tokens should be received and lastly approve the transactions. Each step along the way increases the overall complexity of the process and creates higher risk to make mistakes.

To put it simply, the problem is not only technical part of interoperability between blockchains, but also the missing unified application-level workflow that coordinates selected transfer steps for the user.

4.2 Technical literature review

4.2.1 Approaches to blockchain interoperability

Interoperability in blockchain systems has been studied and also implemented with several approaches. Its main goal is to allow assets and messages to be moved between different blockchain environments that were originally independent systems. Solutions that exist nowadays, are different in the type of problem they solve. Some protocols are focused on fast transfer of assets inside one ecosystem while others are focusing mainly on secure communication between heterogeneous blockchains.

Protocol which is optimized for speed of the transfer often relies on the relayers or liquidity providers. Transfer speed improves user experience but also introduces additional dependencies. On the other hand, more security-focused solutions may

use light clients or cryptographic proofs, which can reduce trust assumptions but increase complexity and also execution time. This shows, that no single protocol solves all interoperability problems in the same way.

4.2.2 Interoperability inside the Ethereum ecosystem

Interoperability within the Ethereum ecosystem is closely connected to the relationship between Layer 1 and Layer 2 networks. Layer 2 networks were implemented to improve scalability and reduce transaction costs, but on the other hand, they also create the need for reliable asset transfers between layers.

Protocols such as **Across Protocol** and **Hop Protocol** solve this part of the problem. Their main goal is to make transfers inside the Ethereum ecosystem faster for the user. Across Protocol uses relayers, where the user defines transfer information and relayers provide liquidity on the destination chain. Hop Protocol is similar to Across because both protocols improve the same aspect of the transfer.

Since these two protocols operate only within the Ethereum ecosystem, they are not able to transfer tokens between different heterogeneous blockchains.

4.2.3 Interoperability between Ethereum and Polkadot

Because Ethereum and Polkadot use different architecture and execution models, the interoperability between them is also more complex. Ethereum uses an account-based smart contract environment, where Polkadot architecture is based on relay chain and parachain. Because of this, transfers between these ecosystems require different interoperability mechanisms than transfers between Ethereum Layer 2 networks.

Snowbridge is a protocol focused on transfers between Ethereum and Polkadot.

It uses verification based on light-client, which allows one ecosystem to verify information from the other without relying only on a trusted third party. This way the security is improved compared to bridges based on centralized validators.

The main limitation of Snowbridge is that it mainly solves the Ethereum to Polkadot part of the route. The problem is, if asset has to start on Layer 2 chain, it may need to be first transferred to Layer 1 before Snowbridge can execute the transfer.

4.2.4 Related interoperability solutions

Other interoperability solutions address similar problems with different trade-offs. **Celer cBridge** is a liquidity-oriented bridge focused on fast transfers across supported networks. Its advantage is practical usability and broader network support, but it depends on available liquidity, supported routes, and the assumptions of the Celer State Guardian Network.

Hyperbridge represents a more security-oriented approach. It focuses on cross-chain communication using cryptographic verification mechanisms, including consensus and state proofs. This can reduce trust assumptions and support heterogeneous blockchain communication, but it is mainly focused on protocol-level interoperability rather than simplifying the end-user workflow.

A workflow-oriented example was **TurtleV2**, developed by Velocity Labs. It aimed to simplify token movement in the Polkadot ecosystem by combining existing mechanisms into a unified user-facing process. It supported Snowbridge-based transfers between Ethereum and Polkadot and later added XCM transfers between Polkadot parachains. TurtleV2 was later discontinued and is no longer operating.

4.2.5 Summary of limitations

The reviewed solutions show that interoperability is already addressed from several perspectives. Some protocols improve transfer speed inside selected ecosystems while trust-minimized bridges improve security between heterogeneous blockchains. So the limitation is not the lack of interoperability tools, but their limited combination of transfer scenarios and use cases.

During the development of this thesis, the selected scenarios were not available as one continuous workflow covering Layer 2 to Layer 2, Layer 2 to Layer 1, Layer 1 to Polkadot, and combined Layer 2 to Polkadot transfers. The gap addressed in this thesis is application-level coordination of selected tools for continuous Ethereum and Polkadot transfers and providing guidance for the user.

4.3 Solution overview (high level)

The proposed solution is a web-based application layer for simplifying asset transfers across multiple blockchain networks. The main purpose of the solution is not to replace existing interoperability protocols, but to improve how selected transfer possibilities are presented and coordinated for the user. Protocols that exist now, already provide mechanisms for transferring assets between networks, but they are often exposed as separate tools with different interfaces, supported routes, and execution steps. This creates additional complexity, especially when the transfer cannot be completed through one direct action.

The proposed application addresses this problem by creating one guided transfer workflow. Instead of manually selecting a bridge, checking supported networks, and deciding the correct order of steps, the user describes the intended transfer through a simple set of inputs. These inputs include the source network, destina-

tion network, asset, amount, and recipient address. All information besides the recipient address are provided by the proposed application. Based on this information, the application prepares a high-level transfer plan and guides the user through the required process.

4.3.1 Main idea of the solution

The main idea is to separate the user's intention from technical parameters. From the user's point of view, the task is to simply move an asset from one network to another. However, the technical process behind this action may require one or more interoperability mechanisms. Some transfers can be executed directly, while others require an intermediate network and more than one transaction.

The application therefore works as a coordination layer. Instead of creating new mechanisms and protocols, it uses existing interoperability mechanisms and combines them into one user-facing workflow. The value of the proposed solution is in planning, simplifying, and presenting the transfer process in a way that is easier for the user to follow, while combining the advantages of both tools in use.

This approach seems most appropriate because the main difficulty is not the lack of interoperability tools. It is quite the opposite. since there are many tools focusing on different types of the transfers its harder for user to find routes which are technically possible but can be harder to execute because of the knowledge necessary to understand the right path, sequence of steps and which assets can be send to exactly which destination and also which tools supports such actions. The proposed solution reduces this complexity by moving part of the decision-making into the application layer.

4.3.2 Design logic

The design was based on the idea that creating a new interoperability protocol would require solving complex problems, including message verification, liquidity management, security of smart contracts, protocol operation and maintenance in long-term. Based on all these reasons and also the fact that many similar solutions already exist which were developed by bigger teams and fundings, the proposed work focuses on combining already existing solutions and not replacing them.

Before the implementation, available interoperability approaches were reviewed. The approach selected had to support two main types of transfers. The first type is transfers between Ethereum-based networks. The second type is transfer between different ecosystem, where the source and destination networks do not share the same architecture. Another thing to consider was that both transfers had to be working both ways. If not, the proposed solution would be incomplete and would allow transfers to be executed both ways.

4.3.3 Supported transfer scenarios

The proposed solution supports three different groups of transfer scenarios.

- The first group contains transfers between EVM-based networks. These transfers represent homogeneous interoperability because the networks share similar user interaction models, wallet behavior, and address formats.
- The second group contains transfers between Ethereum and the Polkadot ecosystem. These transfers represent heterogeneous interoperability because the two ecosystems use different architectures and different technical models.
- The third group contains combined transfers. These transfers are needed when the source network and destination network are not directly connected

through one simple step. For example, an asset may need to move from a Layer 2 network to an intermediate Layer 1 network before it can continue to another ecosystem. In such a case, the application prepares a multi-step workflow and executes the steps in the correct order.

These transfer scenarios were selected because they use the full scale of the available combinations of transfers which two selected tools can offer. In this way the proposed application has a diverse selection of different types of transfers from different sources to wider selection of destinations.

4.3.4 Workflow and architecture overview

The user selects the source network, destination network, asset, amount, and recipient address. Based on this input, the application checks whether the selected transfer can be processed and creates a transfer plan. This plan shows whether the transfer can be executed as a single step or whether it requires multiple steps. Before the execution, the transfer plan is presented to the user for quick overview. During execution, the application monitors the progress, and continues to the next step only after the previous one is completed. The user still signs all transactions through their own wallet, so the application does not take custody of funds.

4.3.5 Limitations of the proposed solution

The main limitation of this proposed system is the fact that it relays on the capabilities of the used mechanisms. If a selected asset or route is not supported, the application cannot show such combination. In this way this limitation is detected but can't be solved since it cannot create a new missing route which is not already made by one of the other tool.

The other limitation is execution time of multi-step transfers. Since the transac-

tions are dependent on the tools used the transaction times are as fast as the tools them self.

The solution is also affected by the differences between supported ecosystems, such as different wallet types and address formats. The application can make these differences easier to manage but it cannot fully remove them.

4.4 Risk assessment

In this work, the main risks are connected with the application layer rather than with risks that would arise from designing a new interoperability protocol. Since the proposed solution builds on existing interoperability mechanisms, the risk assessment focuses mainly on transfer planning, input validation, dependency on external services, and user interaction during execution.

4.4.1 Risk of wrong transfer plan

The transfer plan is an important part of the proposed solution because it determines which route and protocol will be used for the selected transfer. If the plan was created from incorrect data or invalid route mapping, the transfer could fail or not represent the user's intention. This risk is reduced by limiting the user's choices to supported combinations only. The user first selects the source chain, then the application filters available assets, and after that it displays only valid destination chains. The transfer plan is therefore generated from predefined and protocol-supported routes rather than trusting correctness of user input. Unsupported routes are not allowed to continue to execution.

4.4.2 Risk of wrong address format

The supported transfer scenarios work with different address formats. EVM-based networks use EVM addresses, while transfers to the Polkadot ecosystem use Substrate addresses. If the user enters an address in an incorrect format, the transfer may fail. A more serious case is when the address has a valid format but belongs to a wrong recipient. This cannot be fully detected by the application, because blockchain transactions are usually irreversible. To reduce this risk, the application validates the recipient address according to the selected destination ecosystem. If the address format does not match the destination network, the application does not generate the transfer plan.

4.4.3 Risk of external dependencies

The application depends on several external components, such as protocol APIs, RPC endpoints, wallet providers, and supported route registries. If one of these services is unavailable, changes its behavior, or does not return valid data, the transfer may not be prepared or executed correctly. This risk cannot be fully removed, because the application is built on existing interoperability tools. However, it can be mitigated by checking route availability before execution, stopping the process when required data are missing, and showing clear error messages before the user signs a transaction.

4.4.4 Risk of multi-step transfers

Multi-step transfers comes with additional risk because the second step depends on the successful completion of the first one. For example, a transfer from an Ethereum Layer 2 network to the Polkadot ecosystem, first require a transfer to Ethereum Layer 1. If the first transaction fails, is delayed, or is rejected by the user,

the next step must not continue. This is mitigated by sequential execution. The application starts the next step only after the previous step has been completed successfully.

4.4.5 Risk of user error

Even with a simplified interface, the user still approves transactions through their own wallet. The user may reject a transaction, select a wrong account, enter an unintended amount, or misunderstand why multiple confirmations are required. This risk is reduced by showing the transfer plan before execution and by displaying progress during the workflow. The application also does not take custody of funds, so the final approval of each transaction remains in user's control.

4.5 Experimental reproducibility and integration

This section describes how the implemented solution can be reproduced, configured, built, and evaluated. It also explains how the individual external components are integrated into one application workflow. The submitted implementation is the frontend application located in the `bachelor_thesis` directory.

4.5.1 Implementation environment

The implemented solution is a client-side web application. It does not require a custom backend server, because the application communicates directly with external protocol APIs, blockchain RPC endpoints, and browser wallet providers. This makes the prototype easier to reproduce, since the main requirement is to install the frontend dependencies and provide the required runtime configuration.

The application was implemented using React, TypeScript, and Vite. React is used

for the user interface, TypeScript provides transfer-related structures, and Vite is used as the development and build tool. The main implemented user-facing parts are the transfer page and the balance page. The transfer page is responsible for constructing and executing transfer workflows, while the balance page reads non-zero balances from supported EVM networks and Asset Hub environments. The balance page was implemented as part of the prototype to support faster testing and verification of available funds.

The reproducible dependency state is captured by the `package.json` and `package-lock.json` files in the `bachelor_thesis` directory. The most important libraries include React, TypeScript, Vite, `viem`, `@snowbridge/api`, and `@snowbridge/registry`. The `viem` library is used for interaction with EVM-compatible networks and wallets, while the Snowbridge packages are used for integration with the Ethereum and Polkadot bridge workflow.

The application can be installed and started in a development environment using the following commands:

```
cd bachelor_thesis
cd bp-ui
npm install
npm run dev
```

A production build can be created using:

```
cd bachelor_thesis
cd bp-ui
npm install
npm run build
```

The project was verified by producing a production Vite build from the `bp-ui` di-

rectory. The generated build output can then be used as a static frontend application. Static analysis can also be executed during development, but the production build is the main verification target for confirming that the submitted frontend can be compiled.

4.5.2 Configuration and reproducibility

The application uses Vite environment variables for configuration. These variables are provided through local environment files, for example `.env.local` and `.env`. This separates configuration from source code and prevents sensitive local values from being committed to the repository. Private keys, seed phrases, wallet secrets, or account credentials are not required in the frontend configuration, because all transaction signing is performed by the user's browser wallet.

The most important configuration values are RPC endpoints for the supported EVM networks and optional RPC endpoints for Asset Hub environments. The EVM configuration includes variables for Ethereum Mainnet, Optimism, Base, Arbitrum, Ethereum Sepolia, Base Sepolia, OP Sepolia, Arbitrum Sepolia, and Polygon Amoy. Asset Hub RPC endpoints can also be provided for Westend, Paseo, and Polkadot Asset Hub. If some Asset Hub override variables are not provided, the application can use endpoints obtained from the Snowbridge registry or predefined fallback values where implemented.

The application supports both mainnet and testnet environments. The testnet environment was used throughout the whole development process and final testing. This is important for reproducibility, because the selected environment determines which chains, assets, and bridge routes are available. To be able to repeat the same experiment, the same application version, dependency lockfile, environment selection, RPC configuration, wallet extensions, source chain, destination chain,

token, amount, and recipient address type should be used.

4.5.3 Protocol integration

The main component of the application is the transfer planner. The planner converts one user transfer request into one or more executable transfer steps. The user consists of the selected environment, source and destination chain, token, amount, and also recipient address. Based on inputted values, the planner then decides whether the transfer has to use two protocols in sequence or only use one specific protocol.

For transfers inside the Ethereum ecosystem Across Protocol was implemented. It is used for EVM-to-EVM routes, including transfers between Ethereum Layer 2 networks and transfers between Layer 2 networks and Ethereum Layer 1. The Across integration loads supported routes, derives available chains and tokens, requests executable transaction data, handles ERC-20 approval when required, submits the transfer transaction through the user's wallet, and polls the deposit status when a following step depends on the completed Across fill.

Snowbridge is integrated for transfers between Ethereum or Sepolia and the Polkadot ecosystem through Asset Hub. The Snowbridge integration uses registry data to determine available bridge environments and supported assets. It is responsible for preparing Ethereum-to-Asset-Hub transfers, Asset-Hub-to-Ethereum transfers, validation before submission, and interaction with the required wallet signer.

The important part of the implementation is that these protocol integrations are not exposed to the user as separate tools. Instead, they are combined behind one transfer workflow. For example, an EVM-to-EVM transfer can be represented as one Across step. A transfer from Ethereum Layer 1 to Asset Hub can be represented as one Snowbridge step. A more complex transfer from an Ethereum

Layer 2 network to Asset Hub can be represented as two dependent steps: first an Across transfer from Layer 2 to Layer 1, followed by a Snowbridge transfer from Layer 1 to Asset Hub. In the opposite direction, an Asset Hub to Layer 2 transfer can first use Snowbridge to reach Ethereum Layer 1 and then Across to continue to the selected Layer 2 network.

This design is the main systems-integration contribution of the prototype. The application hides protocol-specific differences behind a unified planning and execution workflow, while still relying on the original protocols for the actual asset transfer.

4.5.4 Wallet and network integration

The application design is non-custodial. It prepares transactions, but the final confirmation remains in the hands of the user and their wallet. This means that the application does not store private keys and does not take custody of user funds.

On both sides of the transfer workflow, the application relies on browser wallet extensions. Other compatible wallets can be used, but during development and testing the main examples were MetaMask for EVM-based networks and Talisman for Substrate and Asset Hub interactions.

On the EVM side, the application uses a browser wallet provider compatible with `window.ethereum`. In practice, the wallet used can be MetaMask or another compatible EVM wallet. The wallet is used to request accounts, switch to the required EVM chain, and submit transactions prepared by the application. This is used for Across transfers, ERC-20 approvals, and the Ethereum side of Snowbridge transfers.

On the Substrate side, the application uses injected Polkadot-compatible browser extensions through `window.injectedWeb3`. In this work, Talisman was used as the main wallet example for Asset Hub source transfers. This allows the application to access an injected signer for Substrate-based transactions. The same principle is used as on the EVM side: the application prepares the transaction, but the user signs it through the wallet extension.

The balance page shows that the application is not used only for sending transactions. It can also read blockchain data. The page reads native balances on EVM networks, ERC-20 token balances, native balances on Asset Hub, and selected foreign asset balances. This is done through RPC connections and available registry data. In this way, the application supports both transaction execution and balance checking. This balance page was only developed to simplify checking if assets arrived and if the sufficient amount was available for testing transfers.

4.5.5 Reproducibility of test scenarios

The experimental transfer scenarios can be repeated by using the same source code, dependency lockfile, environment configuration, wallet setup, and testnet accounts. For each repeated experiment, the selected environment, source chain, destination chain, token, amount, recipient address type, browser, wallet extension, RPC provider, and execution time should be recorded. Transaction hashes should also be stored, because they allow later inspection of the executed blockchain transactions.

The measured execution time should be interpreted carefully. Wallet confirmation time should not be included in the bridge completion time, because it depends on user interaction rather than protocol execution. For this reason, the measured values should focus on the time after the user confirms the transaction and the

application starts monitoring the execution process. This makes the measurement more relevant for evaluating the behavior of the workflow itself.

The goal of the testnet evaluation is not to prove deterministic performance of the underlying protocols. Instead, the goal is to verify that the application can construct a valid transfer plan, execute the required steps in the correct order, wait for dependent steps when needed, and present progress information to the user.

4.5.6 Limitations of reproducibility

The application can be reproduced from the `bachelor_thesis/bp-ui` source code, npm lockfile, and environment configuration. However, exact transfer durations are not fully reproducible, because they depend on external blockchain and bridge infrastructure. Across supported routes can change over time, relayer and liquidity availability can be different, and API responses may also change over time. Snowbridge registry data and runtime behavior may also change as the supported environments evolve.

RPC endpoints represent another limitation. Public RPC providers may be slow, or temporarily unavailable. Wallet extensions can also behave differently across browsers and versions. Testnet faucets, token balances and bridge availability are all external factors that can influence whether a specific experiment can be repeated in exactly the same way.

Dockerization was not implemented as part of the submitted prototype. Since the solution is a client-side application, it can be built directly in a Node.js environment.

The project does not use machine learning models. Therefore, no model training,

model storage, or model reuse process is required for reproducibility.

4.6 Sustainability and environmental impact

Sustainability in this work can be viewed from two different angles. The first being technical sustainability, which means that the application should remain maintainable and usable even when external protocols or supported networks change. The second one is environmental impact, which is connected with the amount of additional infrastructure, computation, and network activity introduced by the proposed solution. Since the application is not a new blockchain protocol, its environmental impact is mainly connected to the operation of the frontend application.

4.6.1 Technical sustainability

From the technical part, the proposed solution was designed to divide user interface pages, transfer planning, protocol execution, catalogs, and network configuration. This internal structure improves the maintainability in the long run, because individual parts can be adjusted in the future without rewriting the whole application from scratch. The transfer planner is separated from protocol executors, so changes in Across or Snowbridge integration can be handled in the executor, catalog, or configuration files.

Another sustainability measure is the use of existing interoperability protocols. Creating and maintaining a new bridge would require security monitoring, gradually upgrading protocol and managing liquidity. The proposed solution solves this by using already developed protocols. This reduces the operational complexity, but on the other hand, the application is dependent on the availability and compatibility of the used protocols, RPC endpoints, wallet providers, and supported

routes.

Reproducibility also contributes to sustainability. The project uses standard web development tools such as React, TypeScript, Vite, and npm dependency management. The dependency state is captured in the `package-lock.json` file, which makes it easier to reproduce the development environment. RPC endpoints are separated through Vite environment variables, while some network and protocol configuration remains in local catalogs. In a longer run, the project would require regular updates, testing after protocol API changes, and maintenance of local chain and protocol configuration.

4.6.2 Environmental impact

The environmental impact of the application itself is limited compared to blockchain infrastructure. The prototype does not run its own validator, relayer, bridge or server. The frontend can be built into static files and hosted using lightweight hosting. This reduces the need for running custom server. The application while running is not making any big computational tasks. Its main tasks are route selection, validation of addresses and plans, transaction preparation, wallet interaction, and monitoring of transfer progress.

However, the application still depends on blockchain transactions, RPC requests, and external protocol infrastructure. These activities have an environmental cost because they use network and computational resources.

The application cannot remove the environmental impact of the used chains or protocols, but it can avoid activity which is not necessary. For example, it should only request route data and status information when needed, avoid excessive polling, and stop monitoring when a transfer is completed. Efficient error handling is also important, because failed or repeated requests can create unnecessary network

load.

The proposed solution also improve sustainability from higher point of view, by guiding the user through supported routes and preventing unsupported combinations. The application can reduce failed transfer attempts caused by wrong configuration or wrong route selection what results in fewer failed attempts, fewer unnecessary transaction submissions, less repeated interaction with external services, and less wasted user time. Although this does not eliminate the environmental impact of blockchain transfers, it helps reduce inefficiency at the application layer which can be avoided.

Overall, the proposed application has a relatively limited environmental impact, because it is a frontend-only coordination layer.

4.7 Employability

During the work on this bachelor thesis, I gained and improved several skills that are useful for the software engineering job market. The most important experience was learning how to integrate external tools and services into one application. I worked with APIs, SDKs, blockchain RPC endpoints, and browser wallet providers. This helped me understand how external services are used in practice and how their data are processed.

The implementation also gave me practical experience with more up-to-date frontend technologies, mainly React, Vite, and TypeScript. These technologies are used in current web development, so the knowledge gained during this work can be applied to frontend or full-stack software engineering positions. TypeScript was especially useful because it helped structure transfer-related data and reduce mistakes when working with more complex application logic.

Another important part of the work was understanding the integration process itself. I had to study technical documentation, test individual protocol functions, connect them with the user interface, and then combine them into one workflow. These skills are most relevant, because real applications often depend on existing libraries, APIs, and third-party systems.

The thesis also helped me work more systematically with version control, which I had used before, but not at such a level. I used Git during development, which helped me manage changes, organize implementation progress, and keep the project in a state which could be reversed anytime if some error occurs. Overall, the thesis gave me practical experience that can support my future work as a junior frontend or full-stack developer or enhance my skills to be more appealing in job market.

4.8 Teamwork, diversity and inclusion

The prototype was developed as an individual bachelor thesis project, but the topic includes several areas that could be divided between multiple team members in a larger development environment.

Since the project combines more areas like frontend development, blockchain protocol integration, wallet interaction, testing, documentation, and user experience design, the work could be divided into several groups according to these areas.

One part of the team could focus on the user interface and user experience. This group would be responsible for the transfer form, preview, progress information, error messages, and balance overview. Since cross-chain transfers are technically complex, this part would be important for making the application understandable for users and reducing the risk of mistakes.

Another part of the team could focus on protocol integration. This role would include working with different protocols, blockchain RPC endpoints, and wallet providers. The main responsibility would be to study protocol documentation, test supported routes, and implement protocol executors.

Final role could be responsible for testing and reproducibility. This would include preparing test scenarios, recording results, checking behavior on supported test-nets, documenting configuration, and verifying that the application can be built and run in a clean environment.

To achieve the final goal, the coordination of members would be needed as anywhere also when it comes to development.

Inclusion in this project is mainly related to making the application understandable for users with different levels of technical knowledge. The workflow should therefore use filtered choices, address validation, a visible transfer plan, and clear error messages to reduce the need for deep blockchain knowledge.

Chapter 5

Conclusion

This thesis addressed the problem of fragmentation in asset transfers across different blockchain networks, layers and ecosystems. Modern use of blockchain is no longer limited to one isolated network and users may interact with Ethereum Layers and external ecosystems such as Polkadot. Although interoperability protocols already exist, they often solve only one part of the transfer process. As a result, difficult transfers may require the user to understand which protocol should be used, which network should be selected, which address format is required, and in what order the transactions should be executed.

The goal of this thesis was not to design a new interoperability protocol or a new bridge. Instead, the work focused on the application layer, where existing interoperability mechanisms can be combined into one workflow. This approach was selected because the main problem was not only the absence of technical interoperability, but also the way in which available interoperability tools are presented and coordinated for the user. The proposed solution acts as a coordination layer that prepares transfer plans, validates user input, and guide the user through the required execution steps.

The implemented prototype was created as a web application which allows the user to select the environment, source network, destination network, asset, amount, and recipient address. Based on these inputs, it creates a transfer plan that determines whether the transfer can be executed in a single-step or multi-step. Across Protocol was integrated for transfers inside the Ethereum ecosystem, while Snowbridge was used for transfers between Ethereum and the Polkadot ecosystem through Asset Hub. The transfer planner combines these integrations so that the user can interact with one workflow instead of switching between different tools.

One of the important properties of the implemented solution is that it remains non-custodial. The application prepares transactions, but the final authorization stays with the user and their wallet. This means that the application does not need to store private keys and does not take control of user funds.

The evaluation was performed in testnet environments using selected transfer scenarios. The measured results showed that transfers inside the Ethereum ecosystem, such as Layer 2 to Layer 2 and Layer 2 to Layer 1 transfers, were completed rather quick, around 25 seconds in the tests made. Transfers involving Snowbridge transactions required more time, around 20 to 22 minutes depending on the scenario. The combined Layer 2 to Asset Hub transfer was not significantly longer than the Ethereum Layer 1 to Asset Hub transfer, because most of the total execution time was caused by the Snowbridge part of the workflow.

These results should not be taken as a general benchmark of the interoperability protocols. The measurements were performed on testnets and depended on external factors such as RPC availability, relayer behavior, bridge processing, wallet interaction, and network conditions, which are different on testnets and different on mainnets. However, the evaluation showed that the application can coordinate selected transfer workflows, execute steps in the correct order, and present the

process to the user.

The work also has several limitations. The application rely on the availability and compatibility of Across Protocol, Snowbridge, RPC endpoints, wallet providers, and supported routes. If an asset, chain, or route is not supported by the protocols, the application cannot create it by itself. The prototype also does not remove risks on protocol level, limitations in the bridges, or delays of blockchain networks. It can reduce complexity for the user, but it cannot control the behavior of external tools.

Future work could focus on extending the number of supported networks, assets, and interoperability protocols. The user interface could also be improved with clearer explanations of fees, expected execution time, and possible failure reasons. Better monitoring and improved error handling with more detailed transfer history would make the application more useful in real-world. A production version would also require longer and better testing with users with different skills and knowledge and long-term maintenance.

Overall, the thesis shows that existing interoperability mechanisms can be combined at the application layer to simplify transfers across both homogeneous and heterogeneous blockchain environments. The main contribution of the work is a combined user-friendly workflow that makes complex cross-layer and cross-chain transfer paths easier to plan, execute, and understand.

Chapter 6

References

Up to 70 references.

Chapter 7

List of figures

Appendix A

Work plan

A.1 Winter semester

During the winter and summer semesters, the progress of the work was regularly consulted with the thesis supervisor on a weekly basis. These consultations were used to discuss the work completed during the previous week, identify current problems, and define the next steps of the project. The first weeks of the winter semester were focused mainly on understanding blockchain technology, analyzing existing interoperability tools, and identifying possible directions for the proposed solution.

- Week 1: Initial communication with the supervisor about the topic and the main idea of the proposed solution.
- Week 2: Collection of information about blockchain interoperability and analysis of various tools that could be used in the project, including reading their documentation.
- Week 3: Analysis of external dependencies, such as wallets and RPC end-

points, with focus on understanding different wallet types and testing their basic behaviour.

- Week 4: Study of Snowbridge documentation as the first selected interoperability tool and collection of necessary test tokens for later experiments.
- Week 5: First Snowbridge implementation in a command-line environment, including working test transfers between Ethereum and Polkadot.
- Week 6: Solving issues connected with changes in the newer version of Snowbridge and adjusting the implementation accordingly.
- Week 9: Search for a second protocol suitable for connecting Ethereum Layer 2 networks and extending the scope of supported transfer scenarios.
- Week 10: Study of frontend development approaches and selection of suitable frontend technologies for the application prototype.
- Week 11: Initial design of the frontend application, including the expected structure, required inputs, and main user-facing functionality.
- Week 12: Completion of the first part of the thesis and preparation of the first short report for evaluation.

A.2 Summer semester

After the winter semester, the work continued with a stronger focus on implementation, integration of the selected protocols, and writing of the thesis. The plan for the summer semester was adjusted according to the results of the previous analysis and consultations with the supervisor.

- Week 1: Meeting with the supervisor and discussion of the work plan for the summer semester.

- Weeks 2–4: Implementation of Across Protocol, which was selected as a suitable tool for transfers inside the Ethereum ecosystem.
- Weeks 5–7: Continuation of the implementation phase, including connection of required dependencies, wallet interaction, RPC endpoints, and integration with the frontend application.
- Week 8: Beginning of work on the scientific part of the bachelor thesis.
- Weeks 9–10: Completion of the implementation and scientific part, followed by a shift of focus towards the conceptual part of the thesis.
- Week 12: Completion of the conceptual part and preparation for the final submission.

A.3 Evaluation of work plan

At the beginning of the work, the main goal was to understand the topic of blockchain interoperability and collect information about existing approaches and tools. This phase was necessary because the original topic was broad and several possible technical directions had to be considered before selecting the final solution. The analysis included available interoperability protocols, wallet requirements, RPC endpoints, and the practical limitations of individual tools.

The next phase focused on selecting tools that could realistically support the proposed solution. During this process, some tools were found to be unsuitable, limited, discontinued, or changed during development. This influenced the final direction of the project and led to the selection of Snowbridge for transfers between Ethereum and Polkadot and Across Protocol for transfers inside the Ethereum ecosystem.

Dividing the work into smaller weekly tasks and consulting progress with the supervisor helped maintain a more systematic development process. The weekly meetings made it easier to identify implementation problems early, adjust the scope of the project, and decide which parts should be prioritized. This was especially important because the work depended on external protocols and testnet environments, where unexpected technical issues could appear.

Starting with practical implementation before finalizing all written parts also helped improve the understanding of the problem. Testing Snowbridge and Across Protocol in practice showed which parts of the workflow were technically possible and which limitations had to be considered. This practical experience later helped with writing the conceptual and scientific parts of the thesis.

Overall, the work plan provided a useful structure for the development of the thesis. Although the exact technical direction changed during the year, the gradual process of analysis, implementation, evaluation, and writing helped keep the project aligned with the main goal: simplifying asset transfers across Ethereum-based networks and the Polkadot ecosystem through a unified application-level workflow.

Appendix B

Installation and User Guide

B.1 Installation overview

The application is implemented as a client-side web application in the `bp-ui` directory. It does not require a custom backend server or database. To run the application locally, the user needs Node.js, npm, configured RPC endpoints, and compatible browser wallets.

B.2 Required software

Before installing the application, the following software and tools should be available:

- **Node.js LTS**, available from <https://nodejs.org>. The npm package manager is installed together with Node.js.
- A modern web browser, such as Google Chrome, Microsoft Edge, or Mozilla Firefox.

- An EVM-compatible browser wallet, for example MetaMask, available from <https://metamask.io>.
- A Substrate/Polkadot-compatible wallet for Asset Hub source transfers, for example Talisman, available from <https://www.talisman.xyz>.

After installing Node.js, the installation can be verified in a terminal by running:

```
node --version
npm --version
```

If both commands return installed versions, Node.js and npm are ready to use.

B.3 Downloading the project

B.3.1 From GitHub repository

The project can be downloaded from the GitHub repository using Git:

```
git clone https://github.com/ADAMUSSO/Bachelor_thesis_V3
cd Bachelor_thesis_V3/bp-ui
```

B.3.2 From ZIP archive

If the project is provided as a ZIP archive, the archive should be extracted first. After extraction, the user should open the `bp-ui` directory, because this directory contains the submitted frontend application.

B.4 Installing dependencies

Frontend dependencies are installed from the `bp-ui` directory. The following command installs the packages defined in `package.json` and locked in `package-lock.json`:

```
npm install
```

The `package-lock.json` file helps reproduce the dependency versions used during development and testing.

B.5 RPC endpoint configuration

The application reads RPC URLs from Vite environment variables. To configure them, create a file named `.env.local` in the `bp-ui` directory:

```
bp-ui/.env.local
```

The EVM RPC configuration can include the following variables:

```
VITE_RPC_ETHEREUM=https://eth-mainnet.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_OP=https://opt-mainnet.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_BASE=https://base-mainnet.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_ARBITRUM=https://arb-mainnet.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_SEPOLIA=https://eth-sepolia.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_BASE_SEPOLIA=https://base-sepolia.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_OP_SEPOLIA=https://opt-sepolia.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_ARBITRUM_SEPOLIA=https://arb-sepolia.g.alchemy.com/v2/YOUR_API_KEY
```

```
VITE_RPC_POLYGON_AMOY=https://polygon-amoy.g.alchemy.com/v2/YOUR_API_KEY
```

These variables are required for the EVM chains that the user wants to use. If a required RPC variable is missing, the application may fail with an error such

as `Missing RPC env`. RPC endpoints can be created using providers such as Alchemy, available from <https://www.alchemy.com>.

For Asset Hub RPCs, the following variables are optional but recommended:

```
VITE_RPC_ASSET_HUB_PASEO=wss://asset-hub-paseo-rpc.n.dwellir.com
VITE_RPC_ASSET_HUB_WESTEND=wss://asset-hub-westend-rpc.n.dwellir.com
VITE_RPC_ASSET_HUB_POLKADOT=wss://asset-hub-polkadot-rpc.n.dwellir.com
```

Asset Hub RPC endpoints can be obtained from Substrate RPC providers, for example Dwellir, available from <https://www.dwellir.com>. For Substrate and Asset Hub connections, WSS endpoints should be used.

Private keys, wallet seed phrases, or account credentials must not be stored in the `.env.local` file. The application uses browser wallets for transaction signing.

B.6 Running the application

The development server can be started from the `bp-ui` directory using:

```
npm run dev
```

After the server starts, Vite prints a local address in the terminal. The application is usually available at:

```
http://localhost:5173
```

A production build can be created using:

```
npm run build
```

The generated production files are stored in the `dist` directory.

B.7 Wallet setup

For EVM transfers, the user has to install and configure MetaMask or another EVM-compatible browser wallet. The wallet is used for account access, chain switching, and transaction signing.

For Asset Hub source transfers, the user needs a Polkadot/Substrate-compatible browser wallet, such as Talisman or the Polkadot.js extension. This wallet is used to provide the Substrate account and signer required for Asset Hub transactions.

The application does not store private keys and does not sign transactions by itself. All transaction approvals are performed directly through the connected browser wallet.

B.8 Testnet tokens

For testnet experiments, the user needs testnet gas tokens on the selected source networks. These tokens are required to pay transaction fees during transfer execution. Testnet tokens can be obtained from public faucets. Alchemy provides faucets for several supported test networks, including Ethereum Sepolia, Base Sepolia, Arbitrum Sepolia, Optimism Sepolia, and Polygon Amoy:

<https://www.alchemy.com/faucets>

For some transfer scenarios, the user also needs a small amount of native testnet ETH. During testing, the Google Ethereum Sepolia Faucet was used to obtain Sepolia ETH for transaction fees. The faucet can provide a small amount of test ETH, for example 0.05 ETH within a 24-hour period, which was sufficient for the tested scenarios. If a transaction fails because of insufficient funds, the user should

check both the transferred token balance and the native gas token balance on the source network.

B.9 User guide and testing workflow

This section describes the practical use of the application during testing. It focuses on the transfer page, supported transfer scenarios, and the balance page used for checking available funds.

B.9.1 Transfer page overview

The transfer page is the main page of the application. It is used to prepare and execute transfers.

The user has to fill in the following fields:

- environment,
- source chain,
- asset,
- amount,
- destination chain,
- recipient address.

The application filters available options automatically. After selecting the source chain, only available assets are shown. After selecting the asset, only supported destination chains are shown. This helps prevent unsupported transfer combinations.

After filling in all required fields, the user clicks **Preview**. The application then

shows the planned transfer steps. If the preview is correct, the user clicks **Submit** and confirms the required transaction or transactions in the connected wallet.

For multi-step transfers, the application continues to the next step only after the previous step is completed.

Figure B.1: Transfer page used for selecting transfer parameters and starting execution.

B.9.2 EVM to EVM transfer

The following procedure is shown on an example transfer from Arbitrum Sepolia to Base Sepolia. The selected asset is ETH and the tested amount is 0.02 ETH.

1. Select the **testnet** environment.
2. Select **Arbitrum Sepolia** as the source chain.
3. Select **ETH** as the transferred asset.
4. Enter the amount **0.002**.

5. Select **Base Sepolia** as the destination chain.
6. Enter a valid EVM recipient address.
7. Click **Preview**.
8. Check the generated transfer plan. For this scenario, the application should create one Across transfer step.
9. Click **Submit**.
10. Confirm the transaction in the connected EVM wallet.
11. Wait until the application displays the completed transfer status.
12. Copy the transaction hash if it is needed for later verification.

The screenshot displays the 'ConnectBridge' interface with a 'Transfer' modal open. The modal has tabs for 'Mainnet' and 'Testnet'. The 'Transfer' section includes a 'Source chain' dropdown set to 'Arbitrum Sepolia', a 'Token' dropdown set to 'ETH', an 'Amount' input field with '0.002', a 'Destination chain' dropdown set to 'Base Sepolia', and a 'Recipient (EVM)' input field with the address '0xc939D662e2870927F113107604363dF5CdF7411a'. Below these fields are 'Preview' and 'Submit' buttons. The 'Transfer Preview' section shows: 'Token: ETH (native)', 'Amount: 0.002', 'Recipient: 0xc939D662e2870927F113107604363dF5CdF7411a', 'Steps: 1', 'ACROSS', 'Arbitrum Sepolia -> Base Sepolia', and 'Wallet required: evm'. The 'Submit progress' section shows a green checkmark, 'Across bridge tx (1)', and a transaction hash '0x6dd0651...cf073e94'. The 'Completed' section is empty. At the bottom, it says 'Ready: 421614 -> 84532'.

Figure B.2: Transfer page with filled information for transfer from EVM to EVM.

B.9.3 Ethereum Layer 1 to Asset Hub transfer

This scenario is used for transfers from Ethereum Layer 1 to Asset Hub. In the testnet environment, the transfer is executed from Ethereum Sepolia to a selected Westend Hub testnet through Snowbridge.

1. Select the **testnet** environment.
2. Select **Sepolia** as the source chain.
3. Select an available asset from the filtered asset list.
4. Enter the transfer amount.
5. Select the required Asset Hub destination chain.
6. Enter a valid Substrate recipient address.
7. Click **Preview**.
8. Check the generated transfer plan. For this scenario, the application should create one Snowbridge transfer step.
9. Click **Submit**.
10. Confirm the required transaction in the connected EVM wallet.
11. Wait until the application displays the completed transfer status.
12. Copy the transaction hash if it is needed for later verification.

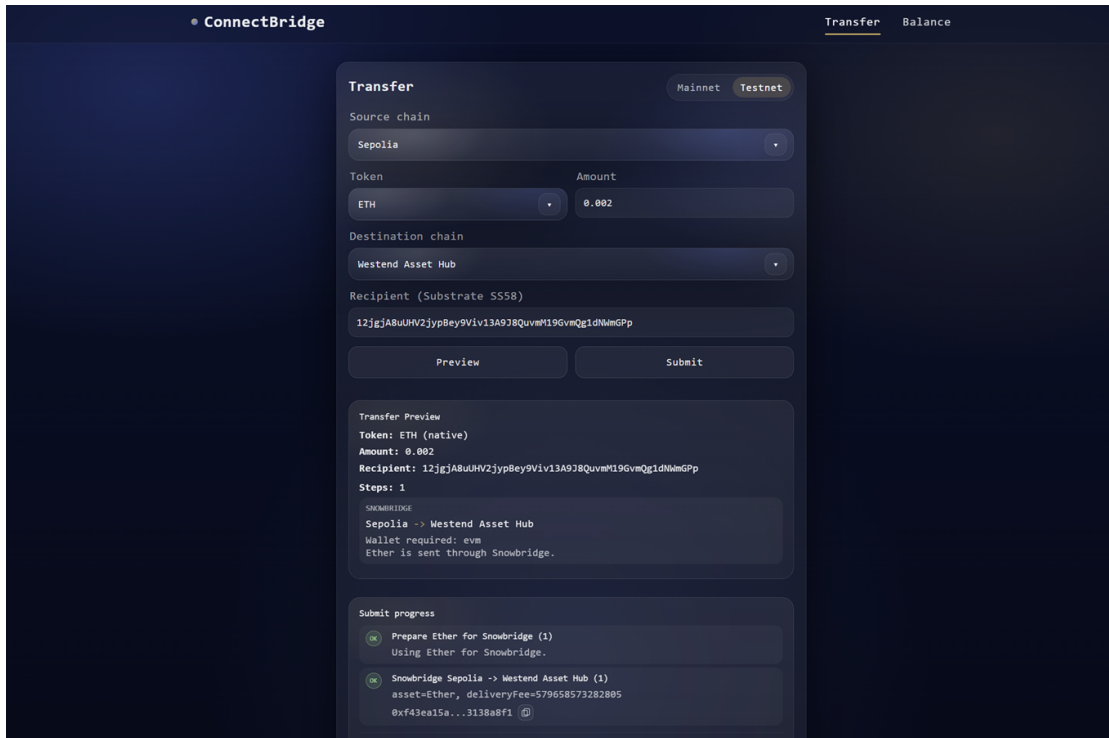


Figure B.3: Transfer page with successful transfer from Ethereum L1 to Westend Asset Hub.

B.9.4 Ethereum Layer 2 to Asset Hub transfer

This scenario is used for transfers from an Ethereum Layer 2 testnet to Asset Hub. In this case, the transfer cannot be executed as one direct step. The application first prepares an Across transfer from the selected Layer 2 network to Ethereum Sepolia. After this step is completed, the application continues with a Snowbridge transfer from Ethereum Sepolia to Asset Hub.

1. Select the **testnet** environment.
2. Select an Ethereum Layer 2 source chain, for example **Arbitrum Sepolia**.
3. Select an available asset from the filtered asset list.

4. Enter the transfer amount.
5. Select the Asset Hub destination chain.
6. Enter a valid Substrate recipient address.
7. Click **Preview**.
8. Check the generated transfer plan. For this scenario, the application should create two steps:
 - Across transfer from Layer 2 to Ethereum Sepolia,
 - Snowbridge transfer from Ethereum Sepolia to Asset Hub.
9. Click **Submit**.
10. Confirm the first transaction in the connected EVM wallet.
11. Wait until the Across transfer is completed.
12. Confirm the required Snowbridge transaction in the EVM wallet.
13. Wait until the application displays the completed transfer status.
14. Copy both transaction hashes if they are needed for later verification.

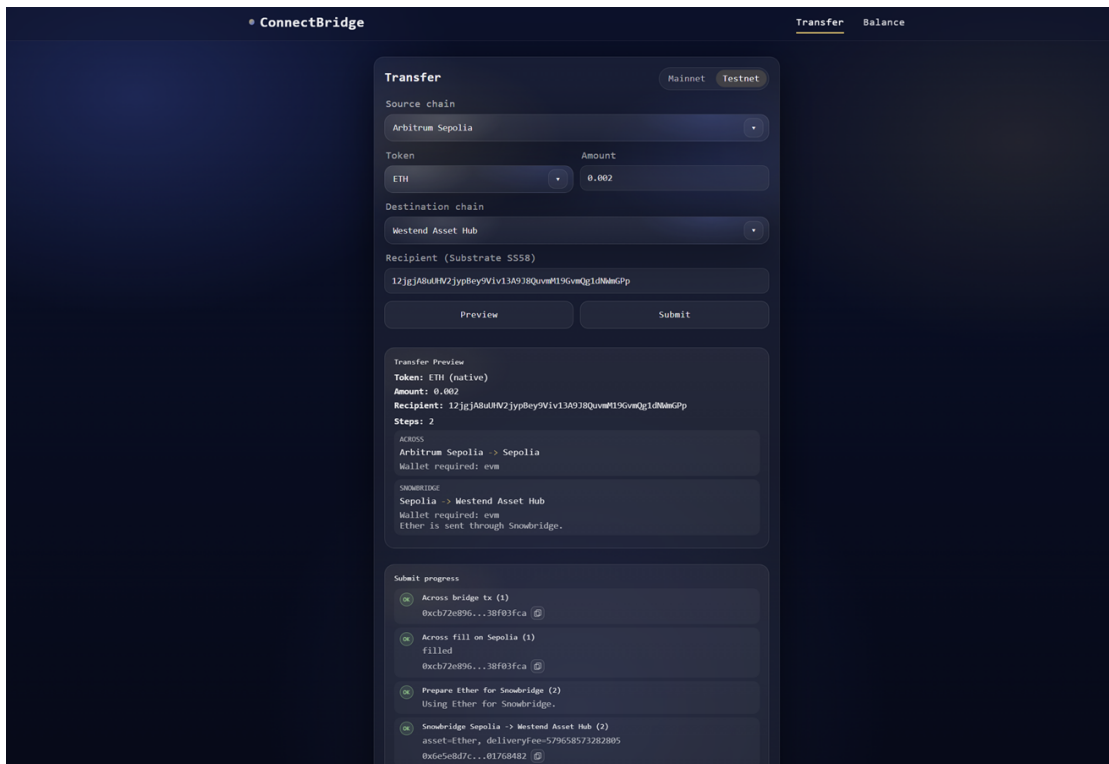


Figure B.4: Transfer page with successful transfer from Ethereum L2 to Westend Asset Hub.

B.9.5 Westend Asset Hub to Ethereum Layer 1 transfer

This scenario is used for transfers from Westend Asset Hub to Ethereum Layer 1. In the testnet environment, the destination Layer 1 network is Ethereum Sepolia. This transfer is executed through Snowbridge in the reverse direction, from Asset Hub back to Ethereum.

1. Select the **testnet** environment.
2. Select **Westend Asset Hub** as the source chain.
3. Select an available asset from the filtered asset list.
4. Enter the transfer amount.

5. Select **Ethereum Sepolia** as the destination chain.
6. Enter a valid EVM recipient address.
7. Click **Preview**.
8. Check the generated transfer plan. For this scenario, the application should create one Snowbridge reverse transfer step.
9. Click **Submit**.
10. Confirm the transaction in the connected Substrate/Polkadot wallet.
11. Wait until the application displays the completed transfer status.
12. Record the transaction hash if it is needed for later verification.

In this scenario, the source chain is not an EVM network. Therefore, the transaction is signed through a Substrate-compatible wallet, such as Talisman. The recipient address must be an EVM address, because the destination network is Ethereum Sepolia.

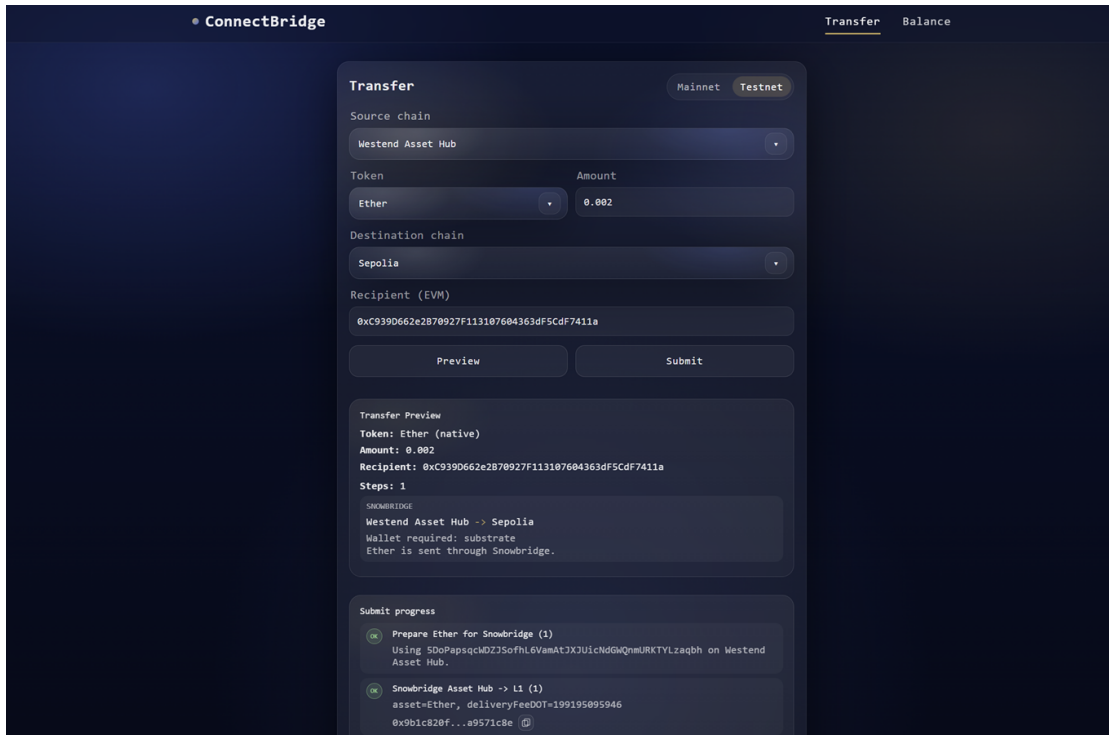


Figure B.5: Example of a transfer from Westend Asset Hub to Ethereum Sepolia.

B.9.6 Westend Asset Hub to Ethereum Layer 2 transfer

This scenario is used for transfers from Westend Asset Hub to an Ethereum Layer 2 testnet. In this case, the transfer is executed as a multi-step workflow. The application first prepares a Snowbridge transfer from Westend Asset Hub to Ethereum Sepolia. After the asset arrives on Ethereum Sepolia, the application continues with an Across transfer from Ethereum Sepolia to the selected Layer 2 network.

1. Select the `testnet` environment.
2. Select `Westend Asset Hub` as the source chain.
3. Select an available asset from the filtered asset list.
4. Enter the transfer amount.

5. Select an Ethereum Layer 2 destination chain, for example **Base Sepolia** or **Arbitrum Sepolia**.
6. Enter a valid EVM recipient address.
7. Click **Preview**.
8. Check the generated transfer plan. For this scenario, the application should create two steps:
 - Snowbridge transfer from Westend Asset Hub to Ethereum Sepolia,
 - Across transfer from Ethereum Sepolia to the selected Layer 2 network.
9. Click **Submit**.
10. Confirm the Snowbridge transaction in the connected Substrate/Polkadot wallet.
11. Wait until the asset arrives on Ethereum Sepolia.
12. Confirm the required Across transaction in the connected EVM wallet.
13. Wait until the application displays the completed transfer status.
14. Record transaction hashes if they are needed for later verification.

This transfer requires interaction with both wallet ecosystems. The first step is signed through the Substrate/Polkadot wallet because the source chain is Westend Asset Hub. The second step is signed through the EVM wallet because the asset continues from Ethereum Sepolia to an Ethereum Layer 2 network. The application waits for the first step to finish before starting the Across part of the workflow.

Appendix C

Digital media contents

This appendix describes the contents of the submitted digital media archive. The archive contains the thesis PDF, the source code of the implemented frontend application, and selected figures used in the thesis.

```
./pdfs/  
    Bachelor_thesis_Svitek_Adam.pdf  
./bp-ui/  
    package.json  
    package-lock.json  
    index.html  
    vite.config.ts  
    tsconfig.json  
    tsconfig.app.json  
    tsconfig.node.json  
    eslint.config.js  
./src/  
    main.tsx
```

```
App.tsx
./pages/
  TransferPage.tsx
  BalancePage.tsx
./features/transfer/
  types.ts
  planner.ts
  ./executors/
    acrossSwapExecutor.ts
    snowbridgeExecutor.ts
    snowbridgeReverseExecutor.ts
./catalog/
  types.ts
  acrossCatalog.ts
  snowbridgeCatalog.ts
  evmChainNames.ts
./evm/
  rpcs.ts
  ensureWalletChain.ts
  feeOverrides.ts
./services/
  acrossSwapApproval.ts
  acrossDepositStatus.ts
  balanceLookup.ts
./components/
  Header.tsx
  ComboBox.tsx
```



```
./utils/  
    amount.ts  
./figures/  
    app-overview.png  
    EVM_to_EVM_transfer.png  
    EVM_to_AssetHub_transfer.png  
    westend-to-l1-transfer.png  
    whole_transfer.png
```


Appendix D

Contents of Included CD-ROM

CD-ROM included to the thesis contains following files:

- `/file1` — First file
- `/file2` — Second file